CHAPTER 1

INTRODUCTION

1.1 Background

A college campus is a busy place with hundreds of students moving between classrooms, labs, the library, the canteen and the hostel every day. Small personal items get left behind constantly: ID cards, calculators, water bottles, chargers, umbrellas and notebooks. Most of these items are found by someone within a few hours, but very few of them make it back to their owners.

Today, a student who loses something usually puts a message in a class WhatsApp group or asks the security desk. A student who finds something hands it to the security desk or sticks a notice on a board. Neither side can see what the other has reported, and nothing connects the two.

1.2 Problem statement

There is no single, searchable record of lost and found items on campus, and no way for a finder to verify that the person asking for an item is its real owner. The result is that items stay unclaimed, or are handed to the wrong person.

1.3 Motivation

A simple web application that every student can open on a phone can solve both problems at once. If finders post what they find and owners post what they lose, a keyword search can bring the two together. If the finder asks a question that only the owner could answer, such as the sticker on a bottle or the brand of a charger, the hand-over can be made with confidence.

1.4 Objectives

1. To design a database that records items, claims and messages with the right relationships between them.
2. To build a REST API that enforces who can do what: only the poster decides a claim, only the campus office hides posts.
3. To build a responsive front end that works well on a phone.
4. To verify ownership through a proof question before an item is handed over.
5. To test every main flow automatically.

1.5 Scope of the project

SAMPLE

Sample document by Final Commit for a fictional college. Names in square brackets are placeholders.

The application covers students and one campus office role, and handles posts, search, claims, messages and moderation. Notifications by email or SMS, integration with the college's student database, and storing photos in a separate file store are outside the scope of this version and are discussed as future work in Chapter 7.

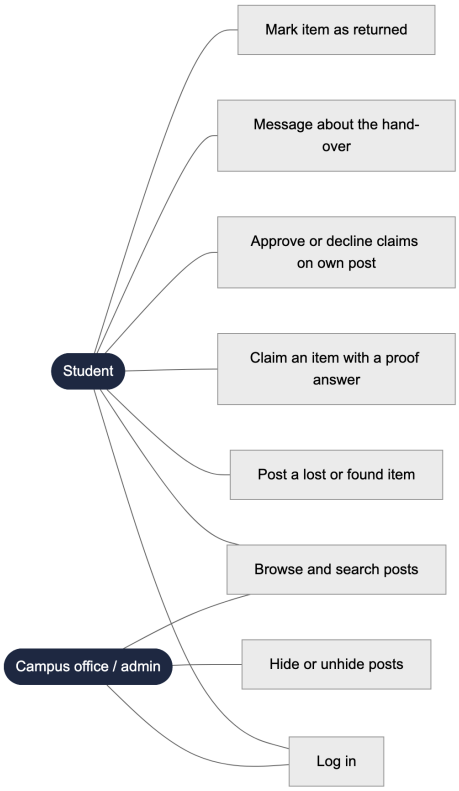


Figure 1.1: Use case diagram

1.6 Methodology

The project follows an iterative approach in four short cycles: database and authentication; posts and search; claims, messages and moderation; and testing and deployment. Each cycle ends with a working demo. The back end is written in Node.js with the Express framework and uses SQLite through Node's built-in *node:sqlite* module, so it has a single dependency. The front end uses plain HTML, CSS and JavaScript modules with no build step. An automated test starts the server with an empty database and checks eighteen behaviours, from refusing a wrong password to making sure the database file cannot be downloaded.

1.7 Organisation of the report

Chapter 2 reviews existing ways of handling lost and found items. Chapter 3 lists the requirements. Chapter 4 describes the system design, with the data flow, entity relationship and sequence diagrams. Chapter 5 covers the implementation, Chapter 6 the testing and results, and Chapter 7 the conclusion and future work.

Sample document by Final Commit for a fictional college. Names in square brackets are placeholders.